

TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN

Modalidad a distancia

Cátedra: Organización Empresarial

TRABAJO PRÁCTICO INTEGRADOR

**Automatización del proceso de rendición y conciliación
de cobros virtuales mediante bot de Telegram**

Organización cliente:

Alejandro Negro S.R.L.

Logística y distribución de alimentos

Alumno:

García Rezusta, Juan Manuel

Comisión 22

Docentes:

Prof. Gabriela Martínez (Titular)

Prof. Carolina Bruno · Prof. Mario Raúl López · Prof. Andrea Ramos

Cohorte Marzo 2026

1. DESCRIPCIÓN DE LA ORGANIZACIÓN Y EL PROBLEMA

1.1 La empresa

Alejandro Negro S.R.L. es una empresa de tipo Sociedad de Responsabilidad Limitada (S.R.L.) dedicada a la logística y distribución de alimentos. Su estructura es de carácter privado y familiar: el socio gerente conduce la operación diaria junto a su hijo, quien oficia de administrativo y responsable operativo del plantel de choferes.

La empresa cuenta con una flota de choferes de reparto que realizan entregas diarias a clientes del rubro gastronómico y minorista. Al finalizar cada entrega, los clientes abonan mediante transferencias bancarias o pagos por Mercado Pago. El chofer no maneja efectivo en estas operaciones: su rol es informar que el cobro se realizó. El sistema no gestiona cobros en efectivo ni notas de crédito, operaciones que forman parte del cierre de caja físico y exceden el alcance de esta solución.

1.2 El problema — proceso AS-IS

Hasta la implementación del sistema, el flujo de rendición de cobros virtuales era completamente manual:

- Cada chofer fotografiaba el comprobante de pago y lo enviaba por WhatsApp al administrativo.
- El administrativo revisaba los mensajes de forma intermitente y volcaba los montos en una planilla de Google Sheets de manera manual.
- Al cierre del día comparaba la planilla con los reportes del banco y de Mercado Pago realizando sumas y comparaciones a mano.
- El proceso era propenso a errores de tipeo, omisiones y duplicados, y dependía de la disponibilidad del administrativo para revisar el grupo de WhatsApp.

2. OBJETIVO DEL SISTEMA

El objetivo fue desarrollar un sistema automatizado que reemplace el flujo manual de WhatsApp, garantizando:

- Registro inmediato y estructurado de cada cobro virtual en cuanto el chofer lo informa.
- Persistencia dual: respaldo local en CSV y sincronización automática con Google Sheets en la nube.
- Conciliación automatizada entre los registros de los choferes y los reportes oficiales del banco y Mercado Pago.
- Trazabilidad completa de cada transacción, con posibilidad de cancelación por el propio chofer en caso de error.

3. MODELADO DEL PROCESO — BPMN 2.0

3.1 Proceso AS-IS

El proceso original consistía en un intercambio informal de mensajes y fotos por WhatsApp entre choferes y el administrativo, sin estructura de datos, sin validaciones automáticas y con alta dependencia del factor humano para la carga y el control de la información.

3.2 Proceso TO-BE

El nuevo proceso se estructura en tres carriles (lanes):

- Chofer: actor humano en campo. Envía mensajes de texto al bot de Telegram con el monto y medio de pago.
- Sistema / Bot: actor automatizado. Parsea el mensaje, valida el monto, clasifica el medio de pago y registra la información.
- Administrativo: actor humano en oficina. Opera el panel de control (Tkinter), carga los archivos fuente y ejecuta la conciliación.

El diagrama BPMN 2.0 completo se adjunta como Figura 1. A continuación se describen las cinco compuertas lógicas del proceso:

Gateway	Condición evaluada	Caminos posibles
GW1 — ¿Monto válido?	El mensaje contiene un número extraíble como monto (regex).	Sí: continúa al GW2. No: el mensaje se ignora silenciosamente.
GW2 — ¿Contiene palabra de banco?	El texto contiene palabras clave: banco, bco, bnc, galicia, bna, santander, bbva, macro, itau, brubank.	Sí: clasifica como transferencia bancaria. No (default): clasifica como Mercado Pago. Incluso un número solo se asume MP.
GW3 — ¿Registro encontrado? (cancelación)	Al cancelar, el sistema busca el msg_id del mensaje respondido en CSV y Sheets.	Sí: elimina el registro y confirma. No: avisa 'no encontré ese pago, capaz ya está borrado'.
GW4 — ¿Todo conciliado?	El sistema cruza cada monto registrado contra los reportes oficiales con tolerancia de centavos.	Sí: continúa al GW5. No: el admin revisa las filas rojas manualmente y re-ejecuta la conciliación.
GW5 — ¿Descargar historial?	El administrativo decide discrecionalmente si es momento de cerrar el lote (semanal, mensual o cuando lo considere oportuno). No es una acción diaria obligatoria.	Sí: ejecuta el cierre de lote, descarga el backup completo a Excel y limpia el Sheets para el próximo período. No: fin de jornada sin cierre, los registros se acumulan para el siguiente día.

Figura 1. Diagrama BPMN 2.0 completo — Archivo adjunto: bpmn_alejandro_negro.png

4. ARQUITECTURA Y STACK TECNOLÓGICO

4.1 Plataforma y lenguaje

La plataforma elegida fue Telegram mediante su API pública de bots. La elección se fundamenta en tres razones: (1) la API es gratuita y no requiere aprobaciones comerciales como WhatsApp Business; (2) la librería python-telegram-bot (v20+) provee handlers y filtros robustos; (3) permite demostración en vivo desde cualquier dispositivo móvil.

El lenguaje de implementación es Python 3.11. La aplicación se ejecuta en el equipo del administrativo mediante un panel de control construido con CustomTkinter, que moderniza la interfaz gráfica nativa de Tkinter.

4.2 Lógica de parseo de mensajes

El parser (función limpiar_entrada en app.py) aplica la siguiente lógica sobre cada mensaje recibido:

- Detecta el monto mediante expresión regular que extrae el primer número del texto, soportando formatos con punto y coma como separadores decimales (ej: '350.000,00' → 350000.0).
- Determina el medio de pago buscando palabras clave de banco en el texto: banco, bco, bnc, bc, galicia, bna, santander, bbva, macro, itau, brubank.
- Si no encuentra ninguna palabra clave de banco, clasifica el pago como Mercado Pago por defecto. Esto significa que 'mp 350000', '350000 mp', 'mercado 350000' y simplemente '350000' son todos interpretados como cobros de Mercado Pago.

4.3 Persistencia de datos — doble respaldo

El sistema implementa persistencia dual para garantizar la integridad ante fallos de red o de la API de Google:

- Respaldo local: archivo CSV generado automáticamente por fecha (respaldo_YYYY-MM-DD.csv). Campos: fecha, hora, chofer, telegram_id, monto, destino, msg_id.
- Base de datos en la nube: Google Sheets accedido mediante la API de Google con autenticación por cuenta de servicio. Se crea automáticamente una pestaña por fecha con subtotales por chofer y formato condicional verde/celeste según estado de conciliación.

En caso de que la pestaña del día esté vacía al reiniciar el sistema, el bot recupera todos los registros del CSV local y los sincroniza automáticamente a Sheets.

4.4 Máquina de estados

El sistema implementa una máquina de estados implícita que representa el ciclo de vida de cada transacción registrada. Cada pago evoluciona a través de distintos estados según las acciones del sistema y del usuario.

Estados del proceso:

• **Registrado / Pendiente:** el cobro fue informado por el chofer y almacenado en el sistema. Se encuentra a la espera de conciliación.

(Implementación: CONCILIADO = FALSE, sin observación)

• **Conciliado automático:** el sistema detecta coincidencia entre el registro y los reportes externos (Banco o Mercado Pago) durante la conciliación.

(Implementación: CONCILIADO = TRUE, observación = "OK")

- **Conciliado manual:** el administrativo valida manualmente un pago que no pudo ser conciliado automáticamente.
(Implementación: `CONCILIADO = TRUE`, observación con valor definido por el usuario)
- **Anulado:** el registro es eliminado por el chofer mediante el comando de cancelación, tanto del almacenamiento local como de la base en la nube.

Esta estructura permite modelar el comportamiento del sistema como una máquina de estados basada en datos persistentes, donde cada transición está determinada por eventos (registro, conciliación, validación manual o cancelación).

4.5 Herramientas de IA utilizadas

El desarrollo del sistema fue asistido de manera intensiva mediante herramientas de inteligencia artificial generativa, principalmente ChatGPT, utilizado como entorno de apoyo iterativo en el diseño, depuración y evolución del sistema. El proceso no fue lineal, sino basado en ciclos sucesivos de prueba, ajuste y refinamiento, donde se construyeron progresivamente las funciones de parseo mediante expresiones regulares, la arquitectura de handlers para la librería python-telegram-bot v20, la lógica de conciliación con tolerancia en montos, el diseño del panel en CustomTkinter y la integración entre persistencia local (CSV) y Google Sheets.

Este enfoque permitió evolucionar desde soluciones parciales hacia un sistema integrado, encastrando progresivamente los módulos hasta lograr un flujo funcional coherente de extremo a extremo.

De forma complementaria, se utilizó Claude como herramienta de apoyo en la revisión y mejora del material documental, especialmente en la estructuración del informe final, la redacción del README del repositorio y el refinamiento de la claridad expositiva del documento.

5. DICCIONARIO DE DATOS

Campo	Tipo	Descripción
fecha	<i>DATE</i>	Fecha del cobro en formato YYYY-MM-DD, ajustada a UTC-3 (horario Argentina).
hora	<i>TIME</i>	Hora del cobro en formato HH:MM:SS, ajustada a UTC-3.
chofer	<i>STRING</i>	Nombre del chofer extraído del perfil de Telegram (first_name) en mayúsculas.
telegram_id	<i>INTEGER</i>	Identificador numérico único del usuario en Telegram. Permite identificar al chofer independientemente del nombre.
monto	<i>FLOAT</i>	Monto del cobro en pesos argentinos. Extraído mediante regex. Soporta formatos con punto y coma como separadores.
destino	<i>STRING</i>	'mp' para Mercado Pago o 'banco' para transferencia bancaria. Determinado por palabras clave. Default: 'mp'.
msg_id	<i>INTEGER</i>	Identificador único del mensaje de Telegram. Permite localizar y eliminar un registro específico al cancelar.
CONCILIADO	<i>BOOLEAN</i>	Estado de conciliación: FALSE (pendiente) o TRUE (conciliado automático o manual).
observacion	<i>STRING</i>	'OK' si fue conciliado automáticamente. Vacío si está pendiente. Valor manual si lo concilió el administrativo.

6. MANUAL DE USUARIO

6.1 Para el chofer — Telegram

El chofer interactúa exclusivamente desde Telegram enviando mensajes al bot del grupo. Todos los formatos siguientes son aceptados:

Formato	Ejemplos	Resultado
mp [monto] / [monto] mp / mercado [monto]	<i>mp 350000 / 350000 mp / mercado 80000</i>	Registra como cobro por Mercado Pago.
[monto] solo (sin palabra clave)	<i>350000</i>	Se asume Mercado Pago por defecto.
[monto] banco / galicia / bna / bco / bnc / santander / bbva / macro / itau / brubank	<i>50000 galicia / bco 30000</i>	Registra como transferencia bancaria.
Responder al mensaje + 'cancelar' (o 'borrar' / 'anular')	<i>Responder + cancelar</i>	Elimina ese registro del CSV y Sheets. El bot confirma con 🗑️.

6.2 Para el administrativo — Panel Tkinter

- ENCENDER BOT: inicia la recepción de mensajes. El botón se pone rojo indicando que el bot está activo.
- ÚLTIMOS MOVIMIENTOS: muestra en tiempo real los últimos 5 registros recibidos, actualizándose cada segundo.
- EJECUTAR CONCILIACIÓN (🏛️): cruza los montos registrados contra los Excel del banco y MP. Los pagos encontrados se marcan en verde. Los no encontrados quedan sin tildar para revisión manual.
- Cerrar la ventana: apaga el bot de forma limpia antes de salir.

7. PRUEBAS DE ROBUSTEZ — CAMINO INFELIZ

Situación	Ejemplo	Respuesta del sistema
Mensaje sin número	<i>"llegue al deposito"</i>	El parser no detecta monto. Mensaje ignorado silenciosamente.
Número solo sin contexto	<i>"350000"</i>	Se asume Mercado Pago por defecto. Registra normalmente.
Monto con formato no estándar	<i>"mp 350.000,00"</i>	El parser normaliza: elimina puntos y convierte coma a punto decimal.
'cancelar' sin responder a un mensaje	<i>Escribe 'cancelar' sin responder</i>	Bot avisa: 'Tenés que responder al pago que querés cancelar.'
Cancelar un pago ya borrado	<i>Responde a mensaje ya eliminado + 'cancelar'</i>	Bot avisa: 'No encontré ese pago. Capaz ya está borrado.'
Sheets vacío al reiniciar el sistema	<i>Bot reiniciado sin datos en la nube</i>	Detecta pestaña vacía y sincroniza automáticamente desde CSV local.
Excel fuente sin pestaña correcta	<i>Admin sube archivo sin hoja 'banco' o 'mp'</i>	Conciliador informa: 'Pestaña no encontrada' o 'Columna de importe no encontrada'.
Excel fuente abierto en otra PC	<i>Archivo .xlsx bloqueado por Excel</i>	Conciliador informa: 'Revisa si está cerrado.'

8. CONCLUSIÓN

El presente trabajo permitió aplicar los conceptos de gestión por procesos sobre un caso real y en producción. El proceso de rendición de cobros virtuales de Alejandro Negro S.R.L. fue modelado mediante la notación BPMN 2.0, identificando con claridad los roles de cada actor, los puntos de decisión y los flujos normales y de excepción.

La automatización implementada eliminó el proceso manual basado en WhatsApp, reduciendo los tiempos de registro, eliminando errores de tipeo y proveyendo al administrativo de una herramienta de conciliación que antes demandaba trabajo intensivo. El sistema se encuentra actualmente en uso real en la empresa. Cabe destacar que el cierre de lote es una acción periódica y discrecional: el administrativo acumula registros durante el período que considere necesario (una semana, un mes o más) y ejecuta el cierre cuando lo decide, sin que esto sea una obligación diaria.

Desde la perspectiva de Organización Empresarial, el caso ilustra cómo una PyME familiar puede optimizar sus procesos administrativos mediante soluciones tecnológicas accesibles. La coherencia entre el modelo BPMN y la lógica implementada garantiza que cualquier modificación futura del proceso pueda ser documentada y comunicada con precisión.

REFERENCIAS

- Object Management Group. (2011). Business Process Model and Notation (BPMN) Version 2.0. OMG.

- python-telegram-bot. (2024). Documentación oficial v20+. <https://python-telegram-bot.org>
- Google. (2024). Google Sheets API v4. <https://developers.google.com/sheets/api>
- gspread. (2024). Librería Python para Google Sheets. <https://gspread.readthedocs.io>